

1. Trực quan hóa dữ liệu có vai trò gì trong phân tích dữ liệu? Tại sao nó quan trọng trong khám phá dữ liệu (EDA)?

Trực quan hóa dữ liệu đóng vai trò quan trọng trong phân tích dữ liệu vì nó giúp biến các con số khô khan thành hình ảnh dễ hiểu, từ đó phát hiện các mẫu, xu hướng và giá trị bất thường. Trong giai đoạn khám phá dữ liệu, còn gọi là EDA, việc sử dụng biểu đồ giúp nhà phân tích nhanh chóng nhận diện phân phối, sự biến thiên và mối quan hệ giữa các biến. Ví dụ, khi phân tích điểm số của học sinh trong một lớp học, việc tạo **histogram** có thể giúp xác định phần lớn học sinh nằm trong khoảng điểm nào và phát hiện những học sinh có điểm quá thấp hoặc quá cao so với mặt bằng chung. Nhờ đó, trực quan hóa dữ liệu trở thành công cụ không thể thiếu trước khi tiến hành các phân tích chuyên sâu.

2. Các loại biểu đồ phổ biến (như histogram, scatter plot, boxplot, bar chart) được sử dụng trong các trường hợp nào?

Mỗi loại biểu đồ phục vụ một mục đích khác nhau dựa trên bản chất dữ liệu và câu hỏi nghiên cứu. **Histogram** thường được dùng với dữ liệu số để quan sát phân phối hoặc tần suất xuất hiện của các giá trị, ví dụ minh họa phân phối điểm số học sinh trong lớp. **Scatter plot** phù hợp để kiểm tra mối quan hệ giữa hai biến số, chẳng hạn như số giờ học và điểm số, giúp nhận diện xu hướng hoặc phân tán dữ liệu. **Boxplot** cung cấp thông tin thống kê cơ bản như median, quartile và phát hiện outlier; ví dụ, có thể dùng boxplot để phát hiện học sinh có điểm ngoại lệ so với cả lớp. **Bar chart** thường dùng cho dữ liệu phân loại, ví dụ minh họa doanh số của từng sản phẩm để so sánh giá trị giữa các nhóm.

3. Làm thế nào để chọn loại biểu đồ phù hợp với đặc điểm của dữ liệu (ví dụ: dữ liệu phân loại, dữ liệu số, dữ liệu thời gian)?

Việc lựa chọn biểu đồ phải dựa trên bản chất của dữ liệu và mục tiêu phân tích. Đối với dữ liệu phân loại, bar chart hoặc pie chart giúp minh họa sự phân bố hoặc tỉ lệ giữa các nhóm; ví dụ, so sánh số lượng học sinh theo từng lớp. Khi dữ liệu là số, histogram hoặc boxplot giúp quan sát phân phối, trong khi scatter plot hữu ích khi muốn xem mối quan hệ giữa hai biến số, ví dụ điểm số và số giờ học. Dữ liệu theo thời gian nên sử dụng line chart, chẳng hạn theo dõi doanh số hàng tháng của một

cửa hàng trong năm. Chọn biểu đồ phù hợp giúp thông tin được truyền tải rõ ràng và chính xác.

4. Sự khác biệt giữa các thư viện trực quan hóa trong Python như Matplotlib, Seaborn và Plotly là gì?

Tiêu chí	Matplotlib	Seaborn	Plotly
Mục đích chính	Tạo các loại biểu đồ cơ bản, kiểm soát chi tiết từng yếu tố	Tạo các biểu đồ thống kê phức tạp, dễ sử dụng trên nền tảng của Matplotlib	Tạo các biểu đồ tương tác cao và động
Tính năng nổi bật	Linh hoạt và tùy chỉnh cao, làm nền tảng cho nhiều thư viện khác	Biểu đồ thống kê đẹp và phức tạp (biểu đồ dải, biểu đồ hộp, v.v.) với cú pháp đơn giản	Đồ thị tương tác, zoom, pan, tooltip, thích hợp cho bảng thông tin và ứng dụng web
Độ phức tạp	Cú pháp có thể phức tạp, đặc biệt cho biểu đồ phức tạp	Cú pháp đơn giản hơn Matplotlib, đặc biệt khi làm việc với Pandas	Cú pháp mạnh mẽ nhưng có thể phức tạp với các tùy chỉnh nâng cao
Ứng dụng	Phân tích dữ liệu cơ bản, tạo biểu đồ tĩnh tùy chỉnh	Phân tích và trực quan hóa dữ liệu thống kê, khám phá dữ liệu	Báo cáo tương tác, bảng thông tin (dashboards), ứng dụng web

5. Những nguyên tắc thiết kế nào cần tuân thủ để tạo ra một biểu đồ trực quan hóa dễ hiểu và hiệu quả?

1. Rõ ràng và đơn giản

- **Loại bỏ sự lộn xộn:** Xóa bỏ các yếu tố không cần thiết như đường lưới, nhãn hoặc màu sắc thừa để tránh gây mất tập trung.
- **Sử dụng ngôn ngữ đơn giản:** Tránh sử dụng quá nhiều thuật ngữ kỹ thuật. Nhãn và chú thích phải rõ ràng, súc tích và dễ đọc.
- **Giữ cho biểu đồ đơn giản:** Nguyên tắc "ít hơn là nhiều hơn" nên được áp dụng. Chỉ đưa vào những gì thực sự nâng cao khả năng hiểu dữ liệu.

2. Lựa chọn biểu đồ phù hợp

- **Chọn đúng loại biểu đồ:** Lựa chọn biểu đồ dựa trên mục tiêu và mối quan hệ của dữ liệu. Ví dụ, dùng biểu đồ cột để so sánh, biểu đồ tròn để hiển thị tỷ lệ.
- **Bắt đầu biểu đồ ở mức 0:** Đối với biểu đồ thanh và cột, hãy bắt đầu từ trục 0 để đảm bảo so sánh chính xác.

3. Sử dụng màu sắc, nhãn và chú thích hiệu quả

- **Giới hạn bảng màu:** Sử dụng màu sắc có chủ đích để làm nổi bật dữ liệu quan trọng, không lạm dụng quá nhiều màu gây nhầm lẫn.
- **Sử dụng nhãn và chú thích:** Đảm bảo nhãn trục và chú thích dễ đọc, ngắn gọn và đặt gần với dữ liệu liên quan.
- **Tuân thủ hệ thống phân cấp:** Sử dụng kích thước và kiểu chữ khác nhau một cách nhất quán để tạo ra hệ thống phân cấp, làm nổi bật các phần quan trọng.

4. Tập trung vào thông điệp

- **Truyền tải một thông điệp chính:** Mỗi hình ảnh trực quan nên tập trung vào một thông điệp cốt lõi. Nếu có nhiều thông điệp, hãy cân nhắc sử dụng nhiều biểu đồ.
- **Tạo tiêu đề hấp dẫn:** Tiêu đề cần ngắn gọn nhưng mạnh mẽ, định hướng cho người xem cách diễn giải biểu đồ.

5. Đảm bảo tính minh bạch và trung thực

- **Trích dẫn nguồn:** Luôn ghi rõ nguồn dữ liệu để tăng uy tín và minh bạch.
- **Trung thực với dữ liệu:** Tránh sử dụng các tùy chọn thiết kế làm sai lệch thông tin hoặc che giấu sự thật.

6. Làm thế nào để tạo một biểu đồ đơn giản như histogram hoặc bar chart bằng Matplotlib? Bạn có thể chia sẻ đoạn code mẫu không?

Việc tạo biểu đồ bằng Matplotlib trong Python rất trực quan. Ví dụ, để tạo một histogram của điểm số học sinh, ta có thể sử dụng đoạn mã sau:

Histogram

```
import matplotlib.pyplot as plt

data = [12, 15, 12, 10, 18, 20, 21, 20, 15, 14, 18, 17]

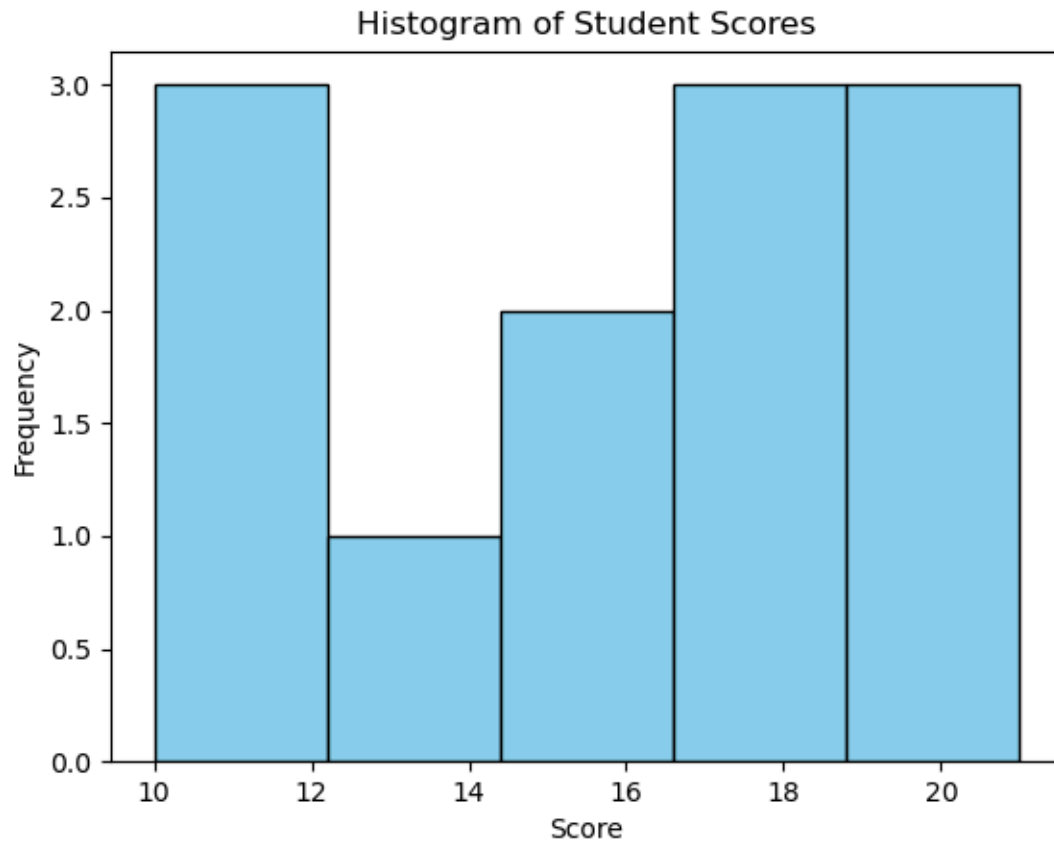
plt.hist(data, bins=5, color='skyblue', edgecolor='black')

plt.title('Histogram of Student Scores')

plt.xlabel('Score')

plt.ylabel('Frequency')

plt.show()
```



Tương tự, để tạo bar chart minh họa doanh số các sản phẩm, ta có thể dùng đoạn mã:

Bar chart

```
import matplotlib.pyplot as plt

categories = ['Product A', 'Product B', 'Product C', 'Product D']
values = [10, 15, 7, 12]

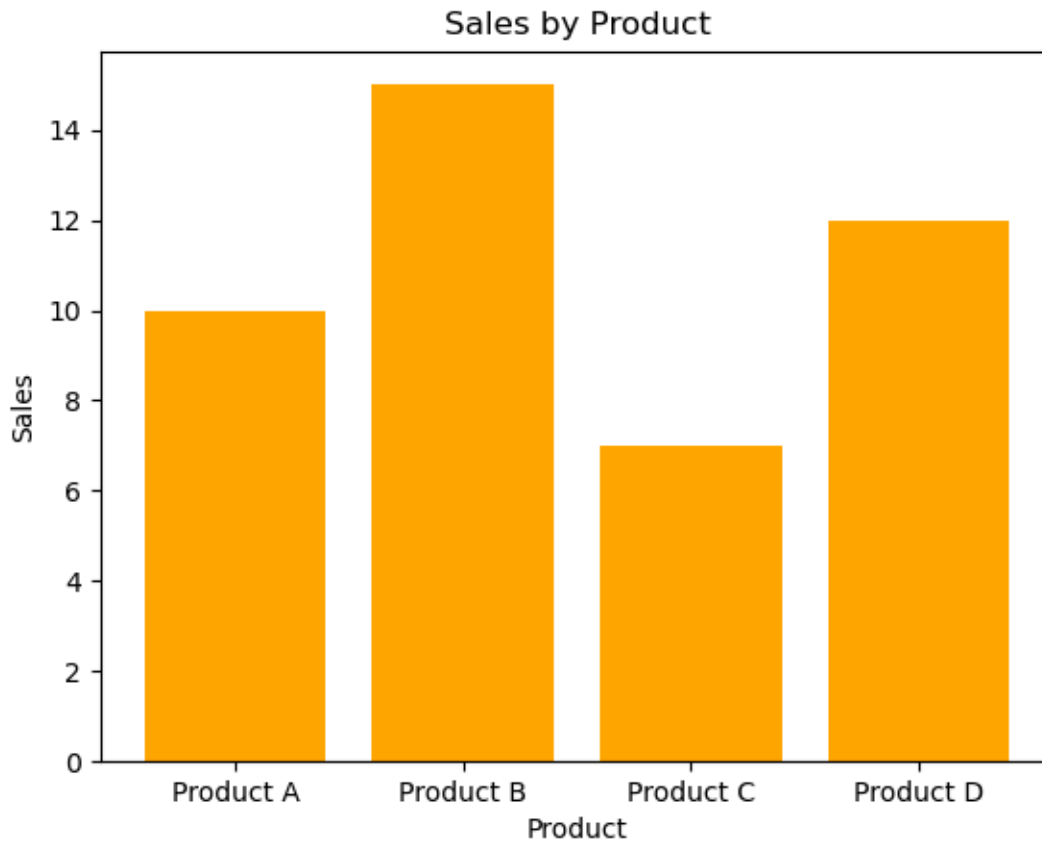
plt.bar(categories, values, color='orange')

plt.title('Sales by Product')

plt.xlabel('Product')

plt.ylabel('Sales')

plt.show()
```



7. Làm thế nào để xuất biểu đồ từ Python ra các định dạng như PNG, PDF hoặc HTML để sử dụng trong báo cáo?

_ Trong phân tích dữ liệu, việc lưu biểu đồ giúp tích hợp trực quan vào báo cáo hoặc trình chiếu. Python hỗ trợ xuất biểu đồ qua các định dạng phổ biến như sau:

+ Xuất biểu đồ dưới dạng hình ảnh (PNG, PDF, SVG...)

_ Sử dụng thư viện **Matplotlib**, ta có thể lưu biểu đồ dưới nhiều định dạng phổ biến như **.png**, **.pdf**, **.svg**, **.jpg** bằng cách sử dụng hàm **savefig()**.

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Tạo dữ liệu mẫu
```

```
data = [10, 20, 30, 40, 50]
```

```
# Vẽ biểu đồ
```

```
sns.histplot(data)
```

```
plt.title("Biểu đồ phân bố dữ liệu")
# Lưu biểu đồ dưới dạng PNG
plt.savefig("bieu_do.png", format='png', dpi=300)
# Lưu biểu đồ dưới dạng PDF
plt.savefig("bieu_do.pdf", format='pdf')

_ format: định dạng tệp cần lưu
_ dpi: độ phân giải (300 phù hợp cho in ấn)
```

+ **Xuất biểu đồ dưới dạng HTML (tương tác)**

_ Đối với biểu đồ tương tác, thư viện **Plotly** là lựa chọn phù hợp. Biểu đồ có thể được lưu dưới dạng tệp **.html** để nhúng vào báo cáo hoặc trình chiếu.

```
import plotly.express as px

# Tạo dữ liệu mẫu
df = px.data.iris()
# Vẽ biểu đồ scatter
fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species")
# Lưu biểu đồ dưới dạng HTML
fig.write_html("bieu_do_tuong_tac.html")

_ Tệp HTML có thể mở bằng trình duyệt hoặc nhúng vào báo cáo web.
```

+ **Lưu biểu đồ vào bộ nhớ (không cần ghi ra file)**

_ Trong một số trường hợp, biểu đồ có thể được lưu vào bộ nhớ dưới dạng đối tượng ảnh để chèn trực tiếp vào báo cáo PDF hoặc Word thông qua các công cụ như **ReportLab**, **docx**, hoặc **nbconvert**.

Kết luận:

_ Việc xuất biểu đồ ra các định dạng như PNG, PDF hoặc HTML giúp người phân tích dữ liệu dễ dàng tích hợp kết quả trực quan vào báo cáo, bài thuyết trình hoặc hệ thống web. Tùy theo mục đích sử dụng (in ấn, chia sẻ, trình chiếu), người dùng có thể lựa chọn định dạng phù hợp để đảm bảo tính chuyên nghiệp và hiệu quả truyền đạt thông tin.